Part 1: Directory Navigation Commands in Linux

Commands covered: **pwd, ls, cd**

1. pwd Command

Theory

The **pwd (Print Working Directory)** command is used to display the complete path of the current directory in which the user is working.

In Linux, everything is organized in a **hierarchical directory structure**, starting from the **root directory (/)**. Every file and directory exists inside this structure.

When a user logs into the system or opens a terminal, they are placed in a **default directory called the home directory**. The pwd command helps the user know their exact position in the filesystem.

This command is very useful when working with multiple directories because it prevents confusion about the current location.

Important Points

- Displays the **absolute path** of the current directory.
- Helps verify the working directory before performing operations like creating, deleting, or copying files.
- Very helpful when navigating deeply nested directories.

Syntax

pwd

Example

pwd

Possible Output

/home/student

This means the user is currently working inside the **student directory located inside /home**.

2. ls Command

Theory

The **ls (list)** command is used to display the contents of a directory. It shows files, directories, and sometimes other objects present in the current location.

It is one of the most frequently used commands in Linux because it allows users to quickly see what files and directories are available.

By default, the ls command lists items in the **current working directory**, but it can also display contents of any specified directory.

Important Points

- Shows files and directories.
- Helps verify whether files were created successfully.
- Can show additional information like file permissions, owner, size, and modification time using options.

Syntax

ls

Example

ls

Possible Output

file1.txt file2.txt testdir

This means the current directory contains two files and one folder.

ls with Detailed Information

Command

ls -l

Theory

The -l option provides a **long listing format** which displays detailed information about each file.

This includes:

- File permissions
- Number of links
- Owner name
- Group name
- File size
- Date and time of last modification
- File name

Example Output

-rw-r--r-- 1 student student 120 Mar 10 10:00 file1.txt

drwxr-xr-x 2 student student 4096 Mar 10 10:02 testdir

Explanation:

Part	Meaning
-rw-r--r--	File permissions
1	Number of links
student	Owner
student	Group
120	File size in bytes
Mar 10 10:00	Last modified time
file1.txt	File name

3. cd Command

Theory

The **cd (change directory)** command is used to move from one directory to another within the Linux filesystem.

This command allows users to navigate through the directory hierarchy easily.

For example, if a user wants to access files inside another folder, they must first move into that folder using the cd command.

Important Points

- Changes the current working directory.
- Used to navigate through folders.
- Can move forward into a directory or backward to a parent directory.

Syntax

cd directory_name

Example 1: Move into a Directory

cd Documents

This command moves the user into the **Documents directory**.

Example 2: Move One Directory Back

cd ..

Theory
The symbol .. represents the **parent directory**.
If the user is inside:
/home/student/Documents
Running cd .. will move them to:
/home/student

Example 3: Move to Home Directory
cd
This command takes the user directly to the **home directory**.

Important Facts About Linux Directories
- / → Root directory (top of the filesystem)
- /home → Contains user home directories
- /bin → Contains essential command binaries
- /etc → System configuration files
- /tmp → Temporary files

Linux follows a **tree-like directory structure** where all directories branch from the root.

Part 2: Directory Creation and File Creation Commands

Commands covered: **mkdir, rmdir, touch**
After learning how to navigate directories (pwd, ls, cd), the next step in a Linux practical is learning how to **create and remove directories and files**. These commands allow users to organize data in the filesystem.

1. mkdir Command
Theory
The **mkdir (make directory)** command is used to create a new directory in Linux.
Directories are used to organize files in a structured way. A directory can contain files as well as other directories, forming a hierarchical structure.
Using mkdir, users can create one or multiple directories at once. It is commonly used when setting up folders for projects, storing files, or organizing data.
Important Points
- Creates a new directory.
- Can create multiple directories at once.
- The user must have **write permission** in the current directory to create a new folder.

Syntax
mkdir directory_name

Example 1: Create a Single Directory
mkdir practical
Explanation
This command creates a new directory named **practical** in the current location.
To verify:
ls

Output might show
practical

Example 2: Create Multiple Directories
mkdir test1 test2 test3
Explanation
This command creates **three directories at the same time**.
Checking the result:
ls
Possible output:
practical test1 test2 test3

Example 3: Create Directory Inside Another Directory
mkdir practical/experiment
Explanation
This creates a directory named **experiment inside the practical folder**.
The directory structure becomes:
practical
└── experiment

2. rmdir Command
Theory
The **rmdir (remove directory)** command is used to delete an **empty directory**.
This command works only if the directory does **not contain any files or subdirectories**. If the directory contains data, Linux will not allow it to be removed using rmdir.
It is useful when a folder is created accidentally or no longer required.
Important Points
- Removes **only empty directories**.
- Cannot delete directories that contain files.
- If a directory has content, other commands like rm -r must be used.

Syntax
rmdir directory_name

Example
rmdir test3
Explanation
This command deletes the directory **test3**, provided it is empty.
Check using:
ls
The folder **test3** will no longer appear.

3. touch Command
Theory
The **touch command** is used to create an empty file in Linux.

Originally, the touch command was designed to **update the timestamp of a file**, but if the file does not exist, Linux automatically creates it.
This command is widely used to quickly create text files, log files, or placeholders for future data.

Important Points

- Creates empty files instantly.
- Can create multiple files at once.
- Updates file timestamps if the file already exists.

Syntax

touch filename

Example 1: Create a File

touch file1.txt

Explanation

This command creates a file named **file1.txt** in the current directory.
Check using:
ls
Possible output:
file1.txt

Example 2: Create Multiple Files

touch file2.txt file3.txt file4.txt

Explanation

This creates three files simultaneously.
Checking with:
ls
Output might be:
file1.txt file2.txt file3.txt file4.txt

Example 3: Create Files in a Specific Directory

touch practical/notes.txt

Explanation

This creates the file **notes.txt inside the practical directory**.
Directory structure becomes:

```
practical
  ├── experiment
  └── notes.txt
```

Important Facts About Files and Directories in Linux

- A **directory** is a container that stores files and other directories.
- Linux filenames are **case-sensitive**.
  File.txt and file.txt are different files.
- Files can contain text, program code, images, or other data.
- The command ls -l can show detailed information about files and directories.

Part 3: File Operation Commands in Linux

Commands covered: **cp, mv, rm**

After creating directories and files, the next important step in Linux is learning how to **copy, move, rename, and delete files**. These operations are essential for managing files efficiently in the filesystem.

1. cp Command

Theory

The **cp (copy)** command is used to copy files or directories from one location to another.

When a file is copied, the **original file remains unchanged**, and a duplicate of the file is created in the specified destination.

This command is commonly used for creating backups, duplicating files, or transferring files between directories.

Important Points

- Copies files from one location to another.
- The original file remains unchanged.
- Can copy both **files and directories**.
- Requires **read permission** for the source file and **write permission** for the destination directory.

Syntax

cp source_file destination

Example 1: Copy File in Same Directory

cp file1.txt copyfile.txt

Explanation

This command creates a duplicate of **file1.txt** and names it **copyfile.txt**.

Checking with:

ls

Output may show:

file1.txt copyfile.txt

Example 2: Copy File to Another Directory

cp file1.txt practical/

Explanation

This copies **file1.txt** into the **practical directory**.

Verify using:

ls practical

Example 3: Copy Multiple Files

cp file1.txt file2.txt practical/

Explanation

Both files are copied to the **practical directory**.

2. mv Command

Theory

The **mv (move)** command is used to move files or directories from one location to another.

Unlike the cp command, the mv command **transfers the file and removes it from the original location**. This command is also used to **rename files and directories**.

Important Points

- Moves files from one directory to another.
- Can also rename files.
- After moving, the original file no longer remains in the previous location.

Syntax

mv source destination

Example 1: Move File to Another Directory

mv file2.txt practical/

Explanation

The file **file2.txt** will be moved into the **practical directory**.

Checking with:

ls practical

Example 2: Rename a File

mv file3.txt newfile.txt

Explanation

This renames **file3.txt** to **newfile.txt**.

Verify using:

ls

Output:

newfile.txt

Example 3: Move Multiple Files

mv file1.txt newfile.txt practical/

Explanation

Both files are moved into the **practical directory**.

3. rm Command

Theory

The **rm (remove)** command is used to delete files in Linux.

Once a file is removed using the rm command, it **cannot be recovered easily**, so it must be used carefully.

The command can delete:

- Single files
- Multiple files
- Entire directories (with additional options)

Important Points

- Permanently deletes files.
- Cannot remove directories without special options.
- Users must have **write permission** to delete files.

Syntax

rm filename

Example 1: Delete a File
rm file1.txt
Explanation
This command deletes **file1.txt** from the current directory.
Verify using:
ls
The file will no longer appear.

Example 2: Delete Multiple Files
rm file2.txt file3.txt
Explanation
Both files will be removed at the same time.

Example 3: Remove Directory with Files
rm -r practical
Explanation
The -r option means **recursive**, which allows deletion of directories along with all their contents.

Example Practical Flow for This Part
Run these commands in sequence during your lab practical:
ls
touch file1.txt
touch file2.txt
touch file3.txt
ls

cp file1.txt copyfile.txt
ls

mkdir practical
cp file2.txt practical/

mv file3.txt practical/

mv copyfile.txt newfile.txt
ls

rm newfile.txt
ls
This demonstrates:

1. File creation
2. File copying
3. Moving files
4. Renaming files
5. Deleting files

Important Facts About File Operations in Linux

- cp creates **duplicate files**.
- mv **moves or renames files**.
- rm **permanently deletes files**.
- Linux file operations depend on **permissions** such as read, write, and execute.

Also remember:

- Linux filenames are **case-sensitive**.
- Hidden files begin with a **dot (.)**.

Example:

.hiddenfile

These files are not visible in normal ls output unless using:

ls -a